

Лабораторная работа № 4

Реализация адаптивности клиентской части и контроль качества с помощью Lighthouse

1. Место работы в курсе

В четвёртой лабораторной работе студент продолжает приложение StudySpace, созданное в лабораторных работах № 2–3. Маршруты App Router, учебное состояние пользователя, галерея, модальное окно, бургер-меню и интерактивная карта сохраняются. Задача новой работы — сделать эти элементы устойчивыми к разным размерам viewport и проверить качество страницы с помощью Lighthouse.

В TheraCity аналогичная задача решается системой контрольных точек, адаптивными CSS-модулями, responsive-изображениями, изменением размеров модальных окон и карты. В StudySpace воспроизводится общий инженерный подход, но не копируются код, дизайн, данные и предметная область TheraCity.

Работа завершает первый блок клиентской части курса. В лабораторных работах № 5–6 к интерфейсу будут добавлены PostgreSQL, Prisma ORM, локальная база данных и серверная авторизация. Поэтому в этой работе учебный DemoAuthProvider по-прежнему не считается защищённой серверной авторизацией.

2. Цель работы

Реализовать адаптивную клиентскую часть веб-приложения на Next.js: создать гибкие макеты для галереи, модального окна, бургер-меню и карты, настроить responsive-изображения, проверить приложение на нескольких ширинах viewport, выполнить Lighthouse-аудит в Chrome DevTools и документированно устранить обнаруженные проблемы.

3. Задачи работы

В ходе выполнения работы студент должен:

- проверить автоматический viewport metadata в App Router;
- применить mobile-first подход и обоснованные контрольные точки;
- использовать относительные размеры, CSS Grid, Flexbox, minmax() и clamp();
- исключить горизонтальную прокрутку страницы на узких экранах;
- сделать сетку карточек адаптивной без дублирования разметки;
- настроить свойство sizes компонента next/image;
- адаптировать модальное окно к ширине и динамической высоте viewport;
- адаптировать бургер-меню и сохранить его клавиатурную доступность;
- обновлять размеры карты после изменения контейнера;
- учесть hover, тип указателя и prefers-reduced-motion;
- проверить интерфейс на нескольких ширинах и при увеличении масштаба;
- собрать production-версию приложения;
- выполнить несколько Lighthouse-запусков с одинаковой конфигурацией;

- сохранить исходные и итоговые показатели;
- исправить обнаруженные проблемы и объяснить внесённые изменения;
- выполнить ручную проверку, которая дополняет автоматический аудит;
- проверить `npm run lint` и `npm run build`.

. Теоретическая часть

4.1 Адаптивный веб-дизайн

MDN определяет responsive web design как подход, при котором страницы хорошо отображаются на экранах разных размеров и разрешений и сохраняют удобство использования. Это не отдельная технология, а набор практик: гибкая сетка, гибкие изображения, современные способы раскладки и media queries. ([MDN: Responsive web design](#))

Фиксированная ширина всей страницы создаёт горизонтальную прокрутку на узком экране и лишнее пустое пространство на широком. Адаптивный интерфейс изменяет расположение и размеры элементов в зависимости от доступного пространства, а не поддерживает отдельную копию страницы для каждого устройства.

Для StudySpace это означает:

- карточки перестраиваются из нескольких колонок в одну;
- текст и отступы плавно уменьшаются;
- модальное окно не выходит за viewport;
- панель меню занимает допустимую ширину;
- карта изменяет высоту и пересчитывает размер контейнера;
- изображения загружаются в размере, соответствующем их фактическому отображению.

4.2 Viewport в Next.js App Router

Без корректного viewport мобильный браузер может использовать виртуальную ширину, рассчитанную для старых desktop-страниц. MDN показывает базовую настройку `width=device-width` и `initial-scale=1`. Она связывает CSS-ширину страницы с шириной устройства и позволяет media queries работать ожидаемо. ([MDN: Responsive web design](#))

В App Router Next.js автоматически добавляет два базовых meta-тега: `charset` и `viewport`. Документация Next.js приводит результат по умолчанию:

```
<meta name="viewport" content="width=device-width, initial-scale=1" />
```

Поэтому вручную дублировать этот тег в `layout.tsx` не требуется. Если нужно настроить, например, `themeColor` или `colorScheme`, App Router поддерживает статический экспорт viewport и функцию `generateViewport` только в Server Components. ([Next.js: Metadata and OG images](#); [Next.js: generateViewport](#))

```
import type { Viewport } from "next";

export const viewport: Viewport = {
```

```
themeColor: "#f6f3ec",
colorScheme: "light",
};
```

Отключать масштабирование пользователя параметрами `maximumScale` или `userScalable` в этой работе нельзя: возможность увеличения страницы необходима для ручной проверки доступности.

4.3 Mobile-first и контрольные точки

MDN описывает `mobile-first` как подход, при котором сначала создаётся простой одноколоночный макет для узкого экрана, а затем `media queries` добавляют более сложную раскладку при появлении достаточного пространства. Контрольная точка, или `breakpoint`, — условие, при котором меняется макет. ([MDN: Responsive web design](#); [MDN: Media query fundamentals](#))

```
.page {
  padding-inline: 16px;
}

@media (min-width: 48rem) {
  .page {
    padding-inline: 32px;
  }
}
```

Контрольные точки выбирают по содержимому: новая точка нужна там, где текущий макет становится тесным или неоправданно растянутым. MDN рекомендует не привязывать `breakpoint` к конкретной модели устройства и использовать относительные единицы. Набор ширин для отчёта является методикой курса, а не перечнем всех существующих устройств.

4.4 CSS Grid, Flexbox и гибкая сетка

`Flexbox` является одномерным способом раскладки по строке или колонке, а `CSS Grid` — двумерной системой строк и колонок. Оба механизма позволяют элементам растягиваться и сжиматься без ручного вычисления координат. ([MDN: CSS layout](#))

Для галереи удобно использовать `Grid` с `minmax()`. Функция `minmax()` задаёт минимальную и максимальную границы размера колонки, а `auto-fit` создаёт столько колонок, сколько помещается в контейнере.

```
.grid {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(min(100%, 15rem), 1fr));
  gap: 1rem;
}
```

Вложенная функция `min(100%, 15rem)` не позволяет минимальной ширине колонки оказаться шире самого контейнера. `Flexbox` подходит для строки действий, заголовка меню и группы фильтров:

```
.actions {
  display: flex;
  flex-wrap: wrap;
}
```

```
    gap: 0.75rem;
}
```

4.5 Относительные размеры, min(), max() и clamp()

Относительные единицы rem, em, %, vw и vh позволяют связывать размер с шрифтом, контейнером или viewport. CSS-функция clamp(MIN, VAL, MAX) ограничивает предпочтительное значение нижней и верхней границами. ([MDN: clamp\(\)](#))

```
.pageTitle {
  font-size: clamp(1.75rem, 4vw, 3rem);
}

.page {
  width: min(72rem, 100%);
  margin-inline: auto;
}
```

clamp() полезна для плавной типографики и отступов, но не заменяет media queries, когда необходимо структурное изменение: например, перенос панели действий или смена направления колонок.

Для модальных окон используется единица dvh — динамическая высота viewport. Она учитывает изменение видимой области на мобильных устройствах, когда панели браузера появляются или скрываются. Ограничение max-height и внутренний overflow-у позволяют прокручивать содержимое окна, не теряя кнопку закрытия.

4.6 Media queries по возможностям устройства и предпочтениям пользователя

Media queries могут проверять не только ширину. MDN описывает media features hover и pointer для определения наличия удобного наведения и точности основного указателя. Эффект, доступный только при наведении, следует включать там, где hover действительно поддерживается.

```
@media (hover: hover) and (pointer: fine) {
  .card:hover {
    transform: translateY(-2px);
  }
}
```

Media feature prefers-reduced-motion сообщает, что пользователь включил системную настройку уменьшения необязательного движения. В этом режиме длительные переходы и анимации следует убрать или существенно сократить. ([MDN: prefers-reduced-motion](#))

```
@media (prefers-reduced-motion: reduce) {
  *,
  *::before,
  *::after {
    scroll-behavior: auto !important;
    animation-duration: 0.01ms !important;
    animation-iteration-count: 1 !important;
    transition-duration: 0.01ms !important;
  }
}
```

4.7 Адаптивные изображения и next/image

Компонент Image выполняет оптимизацию изображений, поддерживает отложенную загрузку и создаёт варианты srcset. Для изображения с fill или изображения, размер которого меняется в CSS, документация Next.js требует осмысленное свойство sizes. Браузер использует sizes, чтобы выбрать подходящий ресурс из srcset. Если sizes отсутствует, браузер предполагает ширину 100vw, что может привести к загрузке слишком большого файла. ([Next.js: Image Component](#))

```
<Image
  src={place.image}
  alt={place.imageAlt}
  fill
  sizes="(max-width: 47.99rem) 100vw, (max-width: 74.99rem) 50vw, 33vw"
  className={styles.image}
/>
```

Контейнер с fill должен иметь позиционирование, а aspect-ratio резервирует устойчивую область до загрузки изображения:

```
.imageWrap {
  position: relative;
  aspect-ratio: 3 / 2;
  overflow: hidden;
}

.image {
  object-fit: cover;
}
```

Свойство preload или высокий fetchPriority применяют только к обоснованному LCP-изображению над первым экраном. Предзагрузка всех карточек конкурирует за сеть и не является оптимизацией.

4.8 Адаптивность модального окна, меню и карты

Адаптивное модальное окно должно сохранять требования лабораторной работы № 3: доступное имя, роль dialog, aria-modal, закрытие Escape, цикл Tab и возврат фокуса. Изменение визуального размера не должно ломать клавиатурную модель.

На узком экране overlay получает меньшие отступы, dialog ограничивается шириной и динамической высотой, а его содержимое прокручивается. Кнопки не должны выходить за область окна.

Панель бургер-меню может занимать всю ширину телефона и ограниченную ширину на desktop. Состояния :focus-visible сохраняются на всех контрольных точках. Закрытая панель не должна оставаться поверх страницы или перехватывать указатель.

Контейнер карты изменяет размер вместе с макетом. ResizeObserver по документации MDN сообщает об изменении размеров элемента. После изменения контейнера API карты необходимо уведомить вызовом map.container.fitToViewport(), предусмотренным API Яндекс Карт. Наблюдатель отключается в cleanup, а карта уничтожается.

4.9 Что проверяет Lighthouse

Lighthouse — открытый автоматизированный инструмент для улучшения качества веб-страниц. Его можно запускать в Chrome DevTools, из командной строки или как Node-модуль. Инструмент получает URL, выполняет набор аудитов и формирует отчёт с объяснениями проблем и ссылками на способы исправления. ([Chrome for Developers: Introduction to Lighthouse](#))

Панель Lighthouse в Chrome DevTools формирует отчёты по категориям:

- Performance — показатели загрузки и выполнения страницы;
- Accessibility — автоматически обнаруживаемые проблемы доступности;
- Best Practices — состояние и общие практики клиентского кода;
- SEO — базовые возможности индексации и поискового представления.

Число категории — агрегированная оценка, а не перечень всех ошибок. В Performance непосредственно в оценку входят метрики; разделы Opportunities и Diagnostics помогают искать причины и влияют на оценку косвенно через изменение метрик. ([Chrome for Developers: Lighthouse performance scoring](#))

4.10 Режимы и воспроизводимость запуска

Официальное руководство Chrome DevTools предлагает сначала сохранить baseline — результат до улучшений. Для проверки загрузки страницы в этой работе используется Mode: Navigation. Device: Mobile изменяет user agent и моделирует мобильный viewport. По умолчанию применяется simulated throttling, которое оценивает поведение при ограниченных мобильных условиях. ([Chrome DevTools: Lighthouse](#))

Результаты могут меняться из-за устройства, фоновых процессов, маршрута сети, расширений браузера и динамического содержимого. Документация Lighthouse прямо предупреждает о вариативности Performance score. Поэтому сравнивать нужно запуски с одной конфигурацией и по возможности в чистом окне Chrome без расширений. ([Chrome for Developers: Lighthouse performance scoring](#))

В этой лабораторной работе выполняются три запуска до и три после изменений. Для таблицы используется медиана трёх значений. Это методика курса для уменьшения влияния случайного запуска, а не официальный проходной балл Lighthouse.

Для анализа запускают production-сборку:

```
npm run lint
npm run build
npm run start
```

Проверка development-сервера и production-сервера отвечает разным задачам. Итоговые показатели Lighthouse фиксируются для production-версии на одном компьютере и в одинаковых условиях.

4.11 Core Web Vitals и лабораторные метрики

Core Web Vitals — подмножество Web Vitals для оценки реального пользовательского опыта. Актуальный набор включает LCP, INP и CLS. Официальные рекомендуемые границы для хорошего пользовательского опыта: LCP не более 2,5 секунды, INP не более 200 миллисекунд и CLS не более 0,1 для 75-го перцентиля загрузок отдельно на мобильных и desktop-устройствах. ([web.dev: Web Vitals](https://web.dev/web-vitals))

- LCP характеризует скорость появления крупнейшего видимого элемента содержимого;
- INP характеризует отзывчивость страницы на взаимодействия;
- CLS характеризует визуальную стабильность и неожиданные сдвиги макета.

Необходимо различать лабораторные и полевые данные. Lighthouse загружает страницу в моделируемой среде без реального пользовательского ввода, поэтому не измеряет INP напрямую. В лаборатории для поиска потенциальных проблем отзывчивости используется TBT как проху-метрика. LCP и CLS доступны в Lighthouse. Полевые значения зависят от реальных устройств, сетей и взаимодействий и не заменяются одним локальным аудитом. ([web.dev: Web Vitals](https://web.dev/web-vitals))

4.12 Автоматическая и ручная проверка

Автоматическая Accessibility-проверка быстро обнаруживает часть проблем: например, отсутствующее alt, некоторые ошибки ARIA и контраст. Официальный курс [web.dev](https://web.dev/automated-accessibility-testing) подчёркивает, что автоматические инструменты не находят все проблемы и могут требовать дополнения ручными и assistive technology тестами. ([web.dev: Automated accessibility testing](https://web.dev/automated-accessibility-testing); [Manual accessibility testing](#))

После Lighthouse студент вручную проверяет:

- последовательность Tab и Shift + Tab;
- видимость фокуса;
- закрытие меню и модального окна клавишей Escape;
- возврат фокуса;
- смысл alt, а не только его наличие;
- отсутствие потери содержимого при масштабе 200%;
- ориентацию portrait и landscape;
- работу мышью, клавиатурой и сенсорным вводом;
- читаемость текста поверх изображений и фонов.

Lighthouse является инструментом контроля и диагностики, но высокий балл не подтверждает полное соответствие всем требованиям доступности, производительности или безопасности.

5. Практическое задание

Этап 1. Подготовка исходной точки

Продолжайте собственный проект лабораторной работы № 3. Не заменяйте предметную область на StudySpace, если ранее была выбрана другая тема.

Перед изменениями проверьте:

- приложение запускается;
- галерея, модальное окно, меню и карта работают;
- `npm run lint` завершается без ошибок;
- `npm run build` создаёт production-сборку.

Сохраните список текущих CSS-файлов и контрольных точек. Зафиксируйте проблемные состояния на ширинах 320, 375, 768, 1024 и 1440 CSS px.

Этап 2. Проверка viewport и базовой страницы

Откройте сгенерированный HTML в DevTools и убедитесь, что Next.js добавил `viewport width=device-width, initial-scale=1`.

При необходимости экспортируйте viewport из `app/layout.tsx` только для `themeColor` и `colorScheme`. Не отключайте масштабирование пользователя.

В `globals.css`:

- установите `box-sizing: border-box`;
- задайте базовые цвета и шрифт;
- ограничьте ширину `main` через `min()`;
- используйте адаптивный `padding` через `clamp()`;
- обеспечьте `max-width: 100%` для `media`-элементов;
- не скрывайте `overflow-x` как способ замаскировать ошибку раскладки.

Этап 3. Mobile-first сетка галереи

Перепишите стили галереи так, чтобы базовый вариант работал в одну колонку. На более широком контейнере Grid должен автоматически добавлять колонки.

Используйте:

```
grid-template-columns: repeat(auto-fit, minmax(min(100%, 15rem), 1fr));
```

Карточка не должна иметь фиксированную ширину. Длинный заголовок и описание должны переноситься, не раздвигая страницу.



Этап 4. Responsive-изображения

Для каждого Image с fill задайте sizes, соответствующее фактической сетке. Сравните вкладку Network при узком и широком viewport и убедитесь, что браузер не обязан всегда выбирать вариант для полной desktop-ширины.

Контейнер изображения должен иметь position: relative и aspect-ratio. Сохраните object-fit: cover и осмысленный alt.

Этап 5. Адаптивное модальное окно

Измените PhotoModal.module.css:

- overlay использует padding, зависящий от ширины;
- dialog имеет width: min(..., 100%);
- max-height рассчитывается от 100dvh;
- содержимое допускает overflow-y: auto;
- изображение сохраняет пропорции;
- строка действий переносится;
- кнопка закрытия остаётся доступной;
- :focus-visible остаётся заметным.

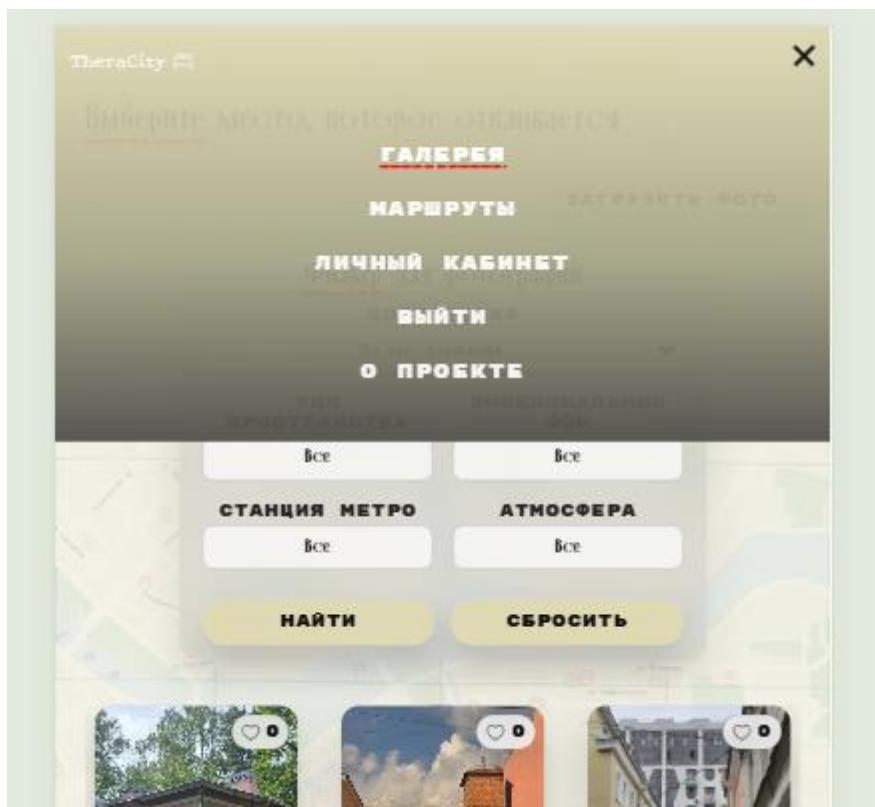
Проверьте окно с коротким и длинным содержимым, в portrait и landscape, а также при масштабе 200%.



Этап 6. Адаптивное бургер-меню

На телефоне панель меню может занимать всю ширину, а на широком экране ограничивается `max-width`. Высота панели должна учитывать 100dvh, а при нехватке высоты содержимое прокручивается.

Не удаляйте `aria-expanded`, `aria-controls`, `role`, `aria-modal`, управление `Tab`, `Shift + Tab` и `Escape`. Проверьте гостевое и пользовательское состояния.



Этап 7. Адаптивная карта

Задайте высоту карты через `clamp()` и уменьшите скругление на узком экране. Добавьте `ResizeObserver` после создания карты. В callback вызывайте `map.container.fitToViewport()`. В cleanup отключите observer и уничтожьте карту.

Проверьте, что изменение ширины окна, открытие DevTools и поворот эмулируемого устройства не оставляют серых областей внутри карты.

Этап 8. Hover, pointer и reduced motion

Перенесите декоративный hover карточки в условие `(hover: hover)` and `(pointer: fine)`. Сенсорный интерфейс не должен зависеть от `hover`.

Добавьте `prefers-reduced-motion: reduce` и отключите необязательные переходы и анимации. Функциональные изменения состояния должны сохраняться.

Этап 9. Ручная матрица адаптивности

Проверьте страницу на ширинах 320, 375, 768, 1024 и 1440 CSS px. Для каждой ширины отметьте:

- количество колонок;
- отсутствие горизонтальной прокрутки;
- читаемость текста;
- доступность всех кнопок;
- работу меню;
- работу модального окна;
- корректное заполнение карты;
- наличие видимого фокуса;
- результат portrait/landscape, если режим доступен.

Отдельно проверьте масштаб браузера 200%. Если элемент не помещается, исправьте причину, а не скрывайте содержимое.

Этап 10. Исходный Lighthouse-аудит

Остановите development-сервер, затем выполните:

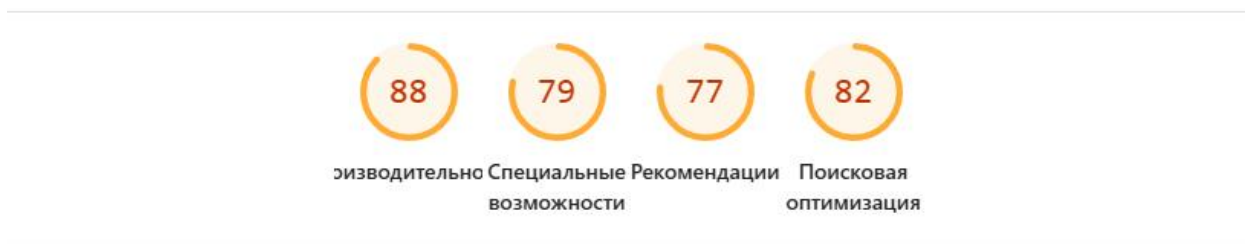
```
npm run lint
npm run build
npm run start
```

Откройте страницу галереи в чистом окне Chrome. В DevTools выберите Lighthouse и одинаковую конфигурацию для всех запусков:

- Mode: Navigation;
- Device: Mobile;
- Categories: Performance, Accessibility, Best Practices, SEO;
- throttling: значение по умолчанию;
- Clear Storage: включено для сценария первого посещения.

Если проверяется маршрут, требующий входа, очистка storage может завершить сеанс. В таком случае зафиксируйте выбранный режим и не меняйте его между сравниваемыми запусками.

Выполните три запуска и сохраните показатели каждой категории, LCP, TBT и CLS. Рассчитайте медиану для таблицы baseline.



Производительность

Значения приблизительные и могут изменяться. [Уровень производительности рассчитывается](#) непосредственно на основании этих показателей. [Показать калькулятор](#)

▲ 0–49 ■ 50–89 ● 90–100



ПОКАЗАТЕЛИ

Развернуть

● First Contentful Paint

0,5 сек.

● Total Blocking Time

40 мс

▲ Speed Index

13,2 сек.

● Largest Contentful Paint

1,1 сек.

● Cumulative Layout Shift

0

[Открыть карту эффективности](#)

Этап 11. Исправление проблем

Изучите failed audits, Opportunities и Diagnostics. Исправьте не менее трёх подтверждённых проблем своего приложения. Возможные направления определяются отчётом, например:

- уточнить sizes и размеры изображений;
- убрать layout shift за счёт зарезервированного aspect-ratio;
- исправить недостаточный контраст;
- добавить доступное имя кнопке;
- исправить структуру заголовков;
- убрать лишнюю клиентскую зависимость;
- отложить необязательный ресурс;

- устранить ошибку Console;
- добавить title и description через Metadata API.

Не изменяйте код только ради числа. Для каждого исправления укажите аудит, причину, изменение и ожидаемый эффект.

Этап 12. Итоговый аудит и проверка

Повторите `npm run lint` и `npm run build`. Запустите ту же production-версию и выполните три Lighthouse-запуска с исходной конфигурацией.

В отчёте сравните медианы до и после. Если отдельный показатель ухудшился или изменился незначительно, объясните возможную причину и подтвердите, что функциональность не была принесена в жертву оценке.

Завершите ручной тест клавиатуры, масштаба и контрольных ширин. Lighthouse-отчёт не заменяет этот этап.

6. Требования к результату

После выполнения работы студент должен предоставить проект, в котором:

- сохранена структура и предметная область предыдущих работ;
- viewport metadata присутствует и масштабирование не отключено;
- базовая раскладка реализована mobile-first;
- отсутствует горизонтальная прокрутка на контрольных ширинах;
- галерея перестраивается без дублирования JSX;
- карточки не имеют фиксированной ширины, ломающей узкий экран;
- Image с fill имеет корректное sizes;
- области изображений резервируют пропорции;
- модальное окно помещается в 100dvh и прокручивает содержимое;
- клавиатурная модель модального окна сохранена;
- бургер-меню адаптировано по ширине и высоте;
- клавиатурная модель меню сохранена;
- карта изменяет размер контейнера без серых областей;
- ResizeObserver отключается в cleanup;
- hover не является единственным способом взаимодействия;
- prefers-reduced-motion уменьшает необязательное движение;
- интерфейс проверен на ширинах 320, 375, 768, 1024 и 1440 CSS px;
- интерфейс проверен при масштабе 200%;
- выполнены три baseline и три итоговых Lighthouse-запуска;
- конфигурация всех сравниваемых запусков одинакова;
- устранены не менее трёх подтверждённых проблем;
- в отчёте приведено сравнение до и после;
- указана версия Chrome и дата проверки;

- `npm run lint` и `npm run build` завершаются без ошибок.

Фиксированный проходной балл Lighthouse в этой работе не задаётся. Оцениваются воспроизводимость проверки, корректность интерпретации, подтверждённые исправления и отсутствие регрессий функциональности.

7. Содержание отчёта

Отчёт должен содержать:

- название и цель работы;
- описание исходного состояния приложения;
- перечень выбранных контрольных точек и их обоснование содержимым;
- фрагмент viewport metadata или объяснение автоматического значения Next.js;
- код адаптивной галереи;
- код sizes для изображений;
- код адаптивного модального окна;
- код адаптивного бургер-меню;
- код ResizeObserver для карты;
- код prefers-reduced-motion;
- ручную матрицу ширин 320, 375, 768, 1024 и 1440 CSS px;
- скриншот страницы на ширине 1440 CSS px;
- скриншот страницы на ширине 375 CSS px;
- скриншот модального окна на узком экране;
- скриншот открытого бургер-меню на узком экране;
- скриншот карты после изменения ширины;
- результат проверки при масштабе 200%;
- версию Chrome, дату и конфигурацию Lighthouse;
- три исходных результата и их медиану;
- снимок исходного Lighthouse-отчёта;
- перечень failed audits, выбранных для исправления;
- описание не менее трёх исправлений;
- три итоговых результата и их медиану;
- снимок итогового Lighthouse-отчёта;
- сравнение Performance, Accessibility, Best Practices и SEO до и после;
- сравнение LCP, TBT и CLS до и после;
- объяснение различия лабораторных и полевых данных;
- результаты `npm run lint` и `npm run build`;
- ответы на контрольные вопросы;
- вывод по работе.

Контрольные вопросы:

- Что означает responsive web design?

- Для чего нужен viewport meta tag?
- Какой viewport Next.js добавляет по умолчанию?
- В чём состоит mobile-first подход?
- Почему breakpoint выбирают по содержимому, а не по модели телефона?
- Чем CSS Grid отличается от Flexbox?
- Как работают auto-fit и minmax() в сетке карточек?
- Для чего используется clamp()?
- Когда нужна единица dvh?
- Что проверяют hover и pointer в media query?
- Для чего применяется prefers-reduced-motion?
- Почему Image с fill должен иметь sizes?
- Как aspect-ratio помогает визуальной стабильности?
- Для чего карте нужен ResizeObserver?
- Что делает map.container.fitToViewport()?
- Какие категории формирует Lighthouse в Chrome DevTools?
- Чем Opportunities и Diagnostics отличаются от метрик Performance?
- Почему Performance score может меняться между запусками?
- Что измеряют LCP, INP и CLS?
- Почему Lighthouse не измеряет INP напрямую?
- Для чего в лаборатории используется TBT?
- Почему локальный Lighthouse-аудит не заменяет полевые данные?
- Почему Accessibility score не заменяет ручную проверку?
- Зачем сравнивать production-сборки с одинаковой конфигурацией?

8. Источники

Теоретическая часть подготовлена по официальной документации MDN Web Docs, Next.js, Chrome for Developers, web.dev и Яндекс Карт. Примеры адаптированы к последовательности курса и учебному приложению StudySpace.

- MDN: Responsive web design — https://developer.mozilla.org/en-US/docs/Learn_web_development/Core/CSS_layout/Responsive_Design
- MDN: Media query fundamentals — https://developer.mozilla.org/en-US/docs/Learn_web_development/Core/CSS_layout/Media_queries
- MDN: CSS media queries — https://developer.mozilla.org/en-US/docs/Web/CSS/Guides/Media_queries
- MDN: CSS flexible box layout — https://developer.mozilla.org/en-US/docs/Web/CSS/Guides/Flexible_box_layout
- MDN: CSS grid layout — https://developer.mozilla.org/en-US/docs/Web/CSS/Guides/Grid_layout
- MDN: minmax() — <https://developer.mozilla.org/en-US/docs/Web/CSS/minmax>
- MDN: clamp() — <https://developer.mozilla.org/en-US/docs/Web/CSS/clamp>
- MDN: aspect-ratio — <https://developer.mozilla.org/en-US/docs/Web/CSS/aspect-ratio>
- MDN: object-fit — <https://developer.mozilla.org/en-US/docs/Web/CSS/object-fit>

- MDN: Viewport concepts — https://developer.mozilla.org/en-US/docs/Web/CSS/Guides/CSSOM_view/Viewport_concepts
- MDN: prefers-reduced-motion — <https://developer.mozilla.org/en-US/docs/Web/CSS/@media/prefers-reduced-motion>
- MDN: ResizeObserver — <https://developer.mozilla.org/en-US/docs/Web/API/ResizeObserver>
- Next.js: Metadata and OG images — <https://nextjs.org/docs/app/getting-started/metadata-and-og-images>
- Next.js: generateViewport — <https://nextjs.org/docs/app/api-reference/functions/generate-viewport>
- Next.js: Image Component — <https://nextjs.org/docs/app/api-reference/components/image>
- Next.js: Image Optimization — <https://nextjs.org/docs/app/getting-started/images>
- Next.js: CLI — <https://nextjs.org/docs/app/api-reference/cli/next>
- Chrome for Developers: Introduction to Lighthouse — <https://developer.chrome.com/docs/lighthouse/overview/>
- Chrome DevTools: Lighthouse — <https://developer.chrome.com/docs/devtools/lighthouse/>
- Chrome for Developers: Lighthouse performance scoring — <https://developer.chrome.com/docs/lighthouse/performance/performance-scoring/>
- web.dev: Web Vitals — <https://web.dev/articles/vitals>
- web.dev: Automated accessibility testing — <https://web.dev/learn/accessibility/test-automated/>
- web.dev: Manual accessibility testing — <https://web.dev/learn/accessibility/test-manual/>
- Yandex Maps JavaScript API: Map — <https://yandex.com/dev/jsapi-v2-1/doc/en/v2-1/ref/reference/Map>

Структура файлов

Показывает файлы, в которых выполняются адаптивные изменения.

```
app/
├── gallery/
│   ├── components/
│   │   ├── PhotoMap.module.css
│   │   ├── PhotoMap.tsx
│   │   ├── PhotoModal.module.css
│   │   ├── PhotoModal.tsx
│   │   ├── PlacesGallery.module.css
│   │   └── PlacesGallery.tsx
│   └── page.tsx
├── ui/
│   └── BurgerMenu/
│       ├── BurgerMenu.tsx
│       └── styles.module.css
├── globals.css
└── layout.tsx
```

Используйте файлы лабораторной работы № 3 и обновите только перечисленные части. Имена предметных сущностей, маршрутов, изображений, текстов и контрольных точек должны соответствовать собственному проекту.

Файл app/layout.tsx

```
// Настраивает метаданные, viewport, провайдер и общую оболочку приложения.

import type { Metadata, Viewport } from "next";
import type { ReactNode } from "react";
import DemoAuthProvider from "../providers/DemoAuthProvider";
import BurgerMenu from "../ui/BurgerMenu/BurgerMenu";
import "../globals.css";

// Задаём заголовок и описание страницы для Metadata API.
export const metadata: Metadata = {
  title: "StudySpace",
  description: "Учебный проект о местах для самостоятельной работы",
};

// Указываем цвет темы и цветовую схему viewport.
export const viewport: Viewport = {
  themeColor: "#f6f3ec",
  colorScheme: "light",
};

// Объявляем основной компонент этого файла.
export default function RootLayout({
  children,
}: Readonly<{ children: ReactNode }>) {
  // Возвращаем JSX-разметку, которую React выведет на странице.
  return (
    <html lang="ru">
      <body>
        <DemoAuthProvider>
          <BurgerMenu />
          {children}
        </DemoAuthProvider>
      </body>
    </html>
  );
}
```

```

    </body>
  </html>
);
}

```

Next.js сам добавляет width=device-width и initial-scale=1. Экспорт viewport выше нужен только для цвета темы и цветовой схемы.

Файл app/globals.css

```

/* Задаёт дизайн-токены, базовую адаптивную страницу и режим уменьшенного движения. */

/* Объявляем общие дизайн-токены приложения. */
:root {
  --page-max: 72rem;
  --page-inline: clamp(1rem, 4vw, 3rem);
  --surface: #ffffff;
  --background: #f6f3ec;
  --text: #17212b;
  --muted: #52606d;
  --border: #c9c4ba;
  --accent: #184e77;
  --focus: #005fcc;
}

*,
*::before,
*::after {
  box-sizing: border-box;
}

html {
  color-scheme: light;
  background: var(--background);
}

/* Задаём базовые параметры страницы и типографики. */
body {
  min-width: 20rem;
  margin: 0;
  background: var(--background);
  color: var(--text);
  font-family: Arial, Helvetica, sans-serif;
}

/* Ограничиваем ширину основного содержимого и выравниваем его по центру. */
main {
  width: min(var(--page-max), 100%);
  margin-inline: auto;
  padding: clamp(4.5rem, 8vw, 6rem) var(--page-inline) 4rem;
}

h1 {
  margin-block: 0 1rem;
  font-size: clamp(1.9rem, 5vw, 3.25rem);
  line-height: 1.1;
  text-wrap: balance;
}

img,
svg,
video,
canvas {
  max-width: 100%;
}

```

```

button,
a,
input,
select,
textarea {
  font: inherit;
}

/* Показываем заметный фокус при клавиатурной навигации. */
:focus-visible {
  outline: 0.25rem solid var(--focus);
  outline-offset: 0.2rem;
}

/* Уменьшаем необязательное движение по системной настройке пользователя. */
@media (prefers-reduced-motion: reduce) {
  *,
  *::before,
  *::after {
    scroll-behavior: auto !important;
    animation-duration: 0.01ms !important;
    animation-iteration-count: 1 !important;
    transition-duration: 0.01ms !important;
  }
}

```

min-width: 20rem задаёт нижнюю учебную границу 320 CSS px. Не добавляйте overflow-x: hidden: он может скрыть ошибку компонента, который шире viewport.

4. Галерея: JSX-фрагмент

```

{/* Выводит адаптивную сетку карточек и responsive-изображения. */}

<div className={styles.grid}>
  {/* Создаём одну карточку для каждого объекта массива. */}
  {studyPlaces.map((place) => (
    <button
      className={styles.card}
      type="button"
      key={place.id}
      onClick={(event) => openFromCard(place, event.currentTarget)}
    >
      <span className={styles.imageWrap}>
        {/* Image выбирает подходящий размер ресурса по свойству sizes. */}
        <Image
          src={place.image}
          alt={place.imageAlt}
          fill
          sizes="(max-width: 47.99rem) calc(100vw - 2rem), (max-width: 74.99rem) 45vw,
22rem"
          className={styles.image}
        />
      </span>
      <span className={styles.cardBody}>
        <span className={styles.title}>{place.title}</span>
        <span className={styles.summary}>{place.summary}</span>
      </span>
    </button>
  )))
</div>

```

Значение sizes должно описывать реальный размер карточки вашего проекта. Не копируйте выражение без проверки в DevTools.

галереи

```
/* Перестраивает галерею, резервирует место изображения и добавляет безопасный hover. */

/* Создаём адаптивную сетку карточек. */
.grid {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(min(100%, 15rem), 1fr));
  gap: clamp(0.9rem, 2.5vw, 1.5rem);
  margin-top: clamp(1.25rem, 4vw, 2rem);
}

/* Оформляем интерактивную карточку объекта. */
.card {
  display: grid;
  min-width: 0;
  overflow: hidden;
  padding: 0;
  border: 1px solid var(--border);
  border-radius: clamp(0.75rem, 2vw, 1rem);
  background: var(--surface);
  color: inherit;
  text-align: left;
  cursor: pointer;
  transition: transform 160ms ease, box-shadow 160ms ease;
}

/* Резервируем область изображения с постоянными пропорциями. */
.imageWrap {
  position: relative;
  display: block;
  aspect-ratio: 3 / 2;
  overflow: hidden;
  background: #d9dde2;
}

.image {
  object-fit: cover;
}

.cardBody {
  display: grid;
  gap: 0.6rem;
  padding: clamp(0.9rem, 3vw, 1.2rem);
}

.title,
.summary {
  min-width: 0;
  overflow-wrap: anywhere;
}

.title {
  font-size: clamp(1.05rem, 2vw, 1.25rem);
  font-weight: 800;
}

.summary {
  color: var(--muted);
  line-height: 1.5;
}

/* Включаем hover-эффект только для точного указателя. */
@media (hover: hover) and (pointer: fine) {
  .card:hover {
    transform: translateY(-0.15rem);
    box-shadow: 0 0.8rem 1.8rem rgb(15 23 42 / 14%);
  }
}
```

```

    }
}

/* Меняем раскладку на соответствующей контрольной ширине. */
@media (min-width: 48rem) {
  .grid {
    grid-template-columns: repeat(auto-fit, minmax(16rem, 1fr));
  }
}

```

C

S

/* Адаптирует модальное окно к ширине, высоте и ориентации viewport. */

S

/* Перекрываем viewport затемнённым фоном. */

МОДАЛЬНОГО ОКНА

```

.overlay {
  position: fixed;
  z-index: 3000;
  display: grid;
  place-items: center;
  padding: clamp(0.5rem, 3vw, 1.5rem);
  background: rgb(15 23 42 / 72%);
}

/* Ограничиваем размеры окна и разрешаем прокрутку длинного содержимого. */
.dialog {
  position: relative;
  width: min(47.5rem, 100%);
  max-height: calc(100dvh - 1rem);
  overflow-y: auto;
  overscroll-behavior: contain;
  padding: clamp(1rem, 4vw, 1.75rem);
  border-radius: clamp(0.75rem, 2vw, 1rem);
  background: var(--surface);
  box-shadow: 0 1.5rem 4.4rem rgb(0 0 0 / 35%);
}

.dialog h2 {
  margin: 0 3rem 1rem 0;
  font-size: clamp(1.4rem, 4vw, 2rem);
  overflow-wrap: anywhere;
}

.close {
  position: absolute;
  top: 0.75rem;
  right: 0.75rem;
  display: grid;
  width: 2.75rem;
  height: 2.75rem;
  place-items: center;
  border: 0;
  border-radius: 50%;
  background: #e8edf2;
  font-size: 1.75rem;
  line-height: 1;
}

.image {
  width: 100%;
  height: auto;
  border-radius: 0.75rem;
  object-fit: cover;
}

.actions {

```

```

display: flex;
flex-wrap: wrap;
gap: 0.75rem;
margin-top: 1rem;
}

.actions > * {
min-height: 2.75rem;
}

/* Меняем раскладку на соответствующей контрольной ширине. */
@media (max-width: 30rem) {
.actions {
display: grid;
}

.actions > * {
width: 100%;
}
}

/* Учитываем небольшую высоту экрана в альбомной ориентации. */
@media (orientation: landscape) and (max-height: 32rem) {
.dialog {
max-height: calc(100dvh - 0.5rem);
}
}

```

C

S

S

бургер-меню

```

/* Адаптирует кнопку и панель бургер-меню с учётом safe area. */

/* Закрепляем кнопку открытия меню поверх страницы. */
.openButton {
position: fixed;
top: 0.5rem; left: 0.5rem; inset-top: 0.5rem;
right: max(0.75rem, env(safe-area-inset-right));
z-index: 2000;
display: grid;
width: 3rem;
height: 3rem;
place-items: center;
border: 1px solid var(--border);
border-radius: 50%;
background: var(--surface);
font-size: 1.5rem;
}

/* Перекрываем viewport затемнённым фоном. */
.overlay {
position: fixed;
inset: 0;
z-index: 2200;
background: rgb(15 23 42 / 55%);
}

/* Размещаем панель меню у правого края viewport. */
.panel {
width: min(26rem, 100%);
height: 100dvh;
margin-left: auto;
overflow-y: auto;
overscroll-behavior: contain;
padding: max(1rem, env(safe-area-inset-top))
max(1rem, env(safe-area-inset-right))
max(1rem, env(safe-area-inset-bottom))
1rem;
}

```

```

    background: var(--surface);
    box-shadow: -1.25rem 0 3.1rem rgb(0 0 0 / 25%);
  }

.header,
.authRow {
  display: flex;
  align-items: center;
  justify-content: space-between;
  gap: 1rem;
}

.list {
  display: grid;
  gap: 0.6rem;
  margin: clamp(2rem, 8vh, 3rem) 0;
  padding: 0;
  list-style: none;
}

.list a {
  display: block;
  min-height: 2.75rem;
  padding: 0.75rem;
  border-radius: 0.6rem;
  color: inherit;
  font-size: clamp(1.05rem, 4vw, 1.25rem);
  text-decoration: none;
}

.list a[aria-current="page"] {
  background: #dbeafe;
  color: #123b5d;
  font-weight: 800;
}

/* Меняем раскладку на соответствующей контрольной ширине. */
@media (max-width: 30rem) {
  .panel {
    width: 100%;
  }

  .authRow {
    align-items: stretch;
    flex-direction: column;
  }
}

```

JSX и клавиатурная логика BurgerMenu из лабораторной работы № 3 сохраняются.

8. Адаптивность карты

Дополните тип карты:

```

// Дополняет тип карты методом пересчёта размеров контейнера.

type YandexMap = {
  container: {
    fitToViewport: () => void;
  };
  geoObjects: {
    add: (placemark: YandexPlacemark) => void;
  };
  destroy: () => void;
};

```

В эффекте создания карты добавьте observer. Переменные объявляются рядом с map:

```
// Хранит ссылки на карту, наблюдатель размеров и признак завершения эффекта.

let disposed = false;
let map: YandexMap | null = null;
let resizeObserver: ResizeObserver | null = null;
```

После создания map и добавления меток:

```
// Следит за размером контейнера и уведомляет карту об изменении viewport.

if (containerRef.current && typeof ResizeObserver !== "undefined") {
  // Создаём наблюдатель за изменением размеров контейнера.
  resizeObserver = new ResizeObserver(() => {
    // Просим API карты пересчитать доступную область.
    map?.container.fitToViewport();
  });
  // Начинаем наблюдение за DOM-контейнером карты.
  resizeObserver.observe(containerRef.current);
}
```

Обновите cleanup:

```
// Освобождает наблюдатель и экземпляр карты при размонтировании компонента.

// В cleanup освобождаем подписки и внешние ресурсы эффекта.
return () => {
  disposed = true;
  // Отключаем наблюдатель размеров.
  resizeObserver?.disconnect();
  // Уничтожаем экземпляр карты и его обработчики.
  map?.destroy();
};
```

Observer создаётся после карты. Не создавайте его на каждом рендере вне useEffect.

С

С

/* Задаёт адаптивную высоту карты и оформление сообщений состояния. */

С

```
.section {
  margin-top: clamp(2rem, 6vw, 3rem);
}
```

карты

/* Задаём явные размеры контейнера интерактивной карты. */

```
.canvas {
  width: 100%;
  height: clamp(18rem, 58vh, 32rem);
  overflow: hidden;
  border: 1px solid var(--border);
  border-radius: clamp(0.75rem, 2vw, 1rem);
  background: #dbe3ea;
}
```

/* Выделяем сообщения о загрузке или ошибке карты. */

```
.status {
  padding: 0.75rem 0.9rem;
  border-left: 0.25rem solid var(--accent);
  background: #e9f2f8;
  overflow-wrap: anywhere;
}
```

```

/* Меняем раскладку на соответствующей контрольной ширине. */
@media (max-width: 30rem) {
  .canvas {
    height: min(65dvh, 24rem);
    border-radius: 0.6rem;
  }
}

```

Команды проверки

```

# Проверяет код, собирает и запускает production-версию.

# После ручной проверки выполняем автоматические проверки проекта.
npm run lint
npm run build
npm run start

```

После запуска откройте собственный маршрут галереи, например <http://localhost:3000/gallery>. В отдельном окне Chrome выполните три baseline и три итоговых Lighthouse-запуска. Сервер останавливается сочетанием Ctrl + C.

11. Шаблон Lighthouse

```

# Шаблон для фиксации одинаковых условий и результатов Lighthouse.

Версия Chrome:
Дата и время:
URL:
Mode: Navigation
Device: Mobile
Throttling:
Clear Storage:

# Ниже записываются три запуска и рассчитывается медиана.
Запуск 1: Performance __; Accessibility __; Best Practices __; SEO __
Запуск 2: Performance __; Accessibility __; Best Practices __; SEO __
Запуск 3: Performance __; Accessibility __; Best Practices __; SEO __
Медиана: Performance __; Accessibility __; Best Practices __; SEO __

LCP: __
TBT: __
CLS: __

Выбранный аудит:
Причина:
Изменение:
Результат повторной проверки:

```

12. Индивидуальная часть

- StudySpace, тексты и маршруты — на собственную предметную область;
- studyPlaces и названия полей — на собственные сущности;
- размеры колонок и контрольные точки — на значения, обоснованные собственным содержанием;
- sizes — на выражение, соответствующее фактической ширине изображений;
- цвета — на палитру проекта с проверенным контрастом;
- URL галереи — на собственный маршрут;

- Lighthouse-значения — только на результаты собственного компьютера и браузера;
- скриншоты — только на изображения собственного приложения и собственных отчётов.

Нельзя выдавать результаты TheraCity, примеры из методических материалов или показатели другого студента за результаты собственного приложения.